

Data Engineering

Coding Interview Guide

Python • SQL • PySpark

Real-World Data Engineering Problems

TABLE OF CONTENTS

PART 1: SQL CODING (Most Common)

- Window Functions & Ranking
- Aggregations & Group By
- Joins & Subqueries
- Date/Time Manipulation
- Data Quality & Deduplication

PART 2: PYTHON CODING

- Data Structures (Lists, Dicts, Sets)
- File Processing (CSV, JSON, Parquet)
- Data Transformation
- Error Handling
- API Integration

PART 3: PYSPARK CODING

- DataFrame Operations
- Transformations vs Actions
- Window Functions
- Performance Optimization
- Real-World Scenarios

PART 4: CODING PATTERNS & TIPS

PART 1: SQL CODING QUESTIONS

What Interviewers Look For in SQL:

- Correct use of JOINS (INNER, LEFT, RIGHT, FULL OUTER)
- Window functions (ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD)
- Aggregations with GROUP BY / HAVING
- Subqueries vs CTEs (WITH clause)
- Date/time manipulation
- Handling NULLs correctly
- Performance awareness (avoid Cartesian products, use indexes conceptually)

SQL Q1: Find Nth Highest Salary

Problem: Given an 'employees' table with columns (id, name, salary, department_id), write a query to find the Nth highest salary (e.g., 2nd highest). **Table: employees**

	id	name	salary	department_id
1	Alice	90000	1	
2	Bob	80000	1	
3	Carol	95000	2	
4	Dave	80000	2	

Solution:

```
-- Solution 1: Using DENSE_RANK (handles ties) WITH ranked_salaries AS ( SELECT salary, DENSE_RANK() OVER
(ORDER BY salary DESC) as rank FROM employees ) SELECT DISTINCT salary FROM ranked_salaries WHERE rank =
2; -- Change to N for Nth highest -- Solution 2: Using LIMIT OFFSET (simpler, no ties handling) SELECT
DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1; -- OFFSET = N-1 -- Solution 3:
Subquery approach SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
```

Key Points:

- DENSE_RANK() handles ties (two people with same salary = same rank)
- RANK() vs DENSE_RANK(): RANK skips numbers after ties (1,2,2,4), DENSE_RANK doesn't (1,2,2,3)
- ROW_NUMBER() assigns unique numbers even for ties (arbitrary ordering)

SQL Q2: Running Total (Cumulative Sum)

Problem: Given a 'transactions' table, calculate running total of sales by date. **Table: transactions**

date	sales
2024-01-01	100
2024-01-02	150
2024-01-03	200

Expected Output:

date	sales	running_total
2024-01-01	100	100
2024-01-02	150	250
2024-01-03	200	450

Solution:

```
-- Solution 1: Window function (BEST)
SELECT date, sales, SUM(sales) OVER (ORDER BY date) as running_total
FROM transactions ORDER BY date;
-- Solution 2: Self-join (AVOID - slow for large data)
SELECT t1.date, t1.sales, SUM(t2.sales) as running_total
FROM transactions t1 JOIN transactions t2 ON t2.date <= t1.date
GROUP BY t1.date, t1.sales ORDER BY t1.date;
```

Interview Tip: Always prefer window functions over self-joins for running totals. Window functions are:

- More readable
- Better performance (single pass vs nested loop)
- Show modern SQL knowledge

SQL Q3: Find Duplicate Records

Problem: Find all duplicate email addresses in a 'users' table and count occurrences. **Table: users**

id	email	name
1	alice@email.com	Alice
2	bob@email.com	Bob
3	alice@email.com	Alicia
4	carol@email.com	Carol

Solution:

```
-- Solution 1: Find duplicate emails with count
SELECT email, COUNT(*) as duplicate_count FROM users GROUP
BY email HAVING COUNT(*) > 1; -- Solution 2: Find all rows with duplicate emails (including row details)
SELECT * FROM users WHERE email IN ( SELECT email FROM users GROUP BY email HAVING COUNT(*) > 1 ); --
Solution 3: Using window function (show which is latest)
SELECT id, email, name, ROW_NUMBER() OVER
(PARTITION BY email ORDER BY id DESC) as row_num FROM users QUALIFY row_num > 1; -- QUALIFY supported in
Snowflake/BigQuery
```

Follow-up Question (Very Common): "How would you deduplicate this table?" Answer: -- Keep only first occurrence (by id) DELETE FROM users WHERE id NOT IN (SELECT MIN(id) FROM users GROUP BY email); -- Or using window function (better for analytics DBs) DELETE FROM users WHERE ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) > 1;

SQL Q4: Self-Join to Find Manager-Employee Relationships

Problem: Given an 'employees' table with manager_id, list all employees with their manager names. **Table:**
employees | id | name | manager_id | |----|-----|-----| | 1 | Alice | NULL | | 2 | Bob | 1 | | 3 | Carol | 1 | | 4 | Dave | 2 |

Solution:

```
-- Solution: Self-join SELECT e.id as employee_id, e.name as employee_name, m.name as manager_name FROM
employees e LEFT JOIN employees m ON e.manager_id = m.id; -- Output: -- | employee_id | employee_name |
manager_name | -- |-----|-----|-----| -- | 1 | Alice | NULL | -- | 2 | Bob |
Alice | -- | 3 | Carol | Alice | -- | 4 | Dave | Bob |
```

Follow-up: "Find employees who earn more than their manager." SELECT e.name as employee_name, e.salary as employee_salary, m.name as manager_name, m.salary as manager_salary FROM employees e JOIN employees m ON e.manager_id = m.id WHERE e.salary > m.salary;

SQL Q5: Calculate Month-over-Month Growth

Problem: Calculate month-over-month revenue growth percentage. **Table: monthly_revenue** | month | revenue |
|-----|-----| | 2024-01-01 | 10000 | | 2024-02-01 | 12000 | | 2024-03-01 | 11000 |

Solution:

```
-- Solution: Using LAG window function
SELECT month, revenue, LAG(revenue) OVER (ORDER BY month) as
prev_month_revenue, revenue - LAG(revenue) OVER (ORDER BY month) as revenue_diff, ROUND( (revenue -
LAG(revenue) OVER (ORDER BY month)) * 100.0 / LAG(revenue) OVER (ORDER BY month), 2 ) as growth_percentage
FROM monthly_revenue;
-- Output:
-- | month | revenue | prev_month_revenue | revenue_diff |
growth_percentage | -- |-----|-----|-----|-----|-----|
| 2024-01-01 | 10000 | NULL | NULL | NULL | -- | 2024-02-01 | 12000 | 10000 | 2000 | 20.00 | -- |
2024-03-01 | 11000 | 12000 | -1000 | -8.33 |
```

Key Window Functions: • LAG(column, offset): Get value from previous row • LEAD(column, offset): Get value from next row • Useful for: time-series comparisons, deltas, growth calculations

SQL Q6: Pivot Data (Rows to Columns)

Problem: Pivot sales data from rows to columns by product category. **Table: sales** | date | category | amount |
|-----|-----|-----| | 2024-01-01 | Electronics | 1000 | | 2024-01-01 | Clothing | 500 | | 2024-01-02 | Electronics |
1200 | | 2024-01-02 | Clothing | 600 | **Desired Output:** | date | Electronics | Clothing | |-----|-----|-----| |
2024-01-01 | 1000 | 500 | | 2024-01-02 | 1200 | 600 |

Solution:

```
-- Solution 1: Using CASE (works in all SQL) SELECT date, SUM(CASE WHEN category = 'Electronics' THEN
amount ELSE 0 END) as Electronics, SUM(CASE WHEN category = 'Clothing' THEN amount ELSE 0 END) as Clothing
FROM sales GROUP BY date; -- Solution 2: Using PIVOT (Snowflake/SQL Server) SELECT * FROM sales PIVOT (
SUM(amount) FOR category IN ('Electronics', 'Clothing') ) as pivoted; -- Solution 3: BigQuery PIVOT SELECT
* FROM sales PIVOT ( SUM(amount) as total_amount FOR category IN ('Electronics', 'Clothing') );
```

SQL Q7: Data Quality - Find Missing Dates

Problem: Find missing dates in a daily sales table (identify data quality issues). **Table: daily_sales** | date | sales |
|-----|-----| | 2024-01-01 | 100 | | 2024-01-02 | 150 | | 2024-01-04 | 200 | -- 2024-01-03 is missing! | 2024-01-05 |
180 |

Solution:

```
-- Solution 1: Using date series (PostgreSQL/Snowflake) WITH date_series AS ( SELECT DATE '2024-01-01' +  
INTERVAL '1 day' * (ROW_NUMBER() OVER () - 1) as date FROM TABLE(GENERATOR(ROWCOUNT => 31)) -- Snowflake  
-- OR in PostgreSQL: SELECT generate_series('2024-01-01'::date, '2024-01-31'::date, '1 day'::interval) )  
SELECT ds.date as missing_date FROM date_series ds LEFT JOIN daily_sales s ON ds.date = s.date WHERE  
s.date IS NULL; -- Solution 2: Using LAG to find gaps SELECT date, LAG(date) OVER (ORDER BY date) as  
prev_date, DATEDIFF('day', LAG(date) OVER (ORDER BY date), date) as days_diff FROM daily_sales QUALIFY  
days_diff > 1; -- BigQuery/Snowflake (or use WHERE in CTE)
```

Real-World Application: This pattern is crucial for data quality monitoring in production pipelines. Use cases: • Daily job monitoring (detect failed loads) • Event stream completeness checks • SLA validation (ensure all expected data arrived)

PART 2: PYTHON CODING QUESTIONS

What Interviewers Look For in Python:

- Clean, readable code (PEP 8 style)
- Proper data structures (when to use list vs dict vs set)
- List comprehensions and generator expressions
- Error handling (try/except)
- File I/O (CSV, JSON, Parquet)
- Working with pandas DataFrames
- Understanding of iterators and generators (memory efficiency)

PYTHON Q1: Parse Nested JSON and Flatten

Problem: Given nested JSON from an API, flatten it into a list of dictionaries. **Input:** { "users": [{"id": 1, "name": "Alice", "address": {"city": "NYC", "zip": "10001"}}, {"id": 2, "name": "Bob", "address": {"city": "LA", "zip": "90001"}}] } **Output:** [{"id": 1, "name": "Alice", "city": "NYC", "zip": "10001"}, {"id": 2, "name": "Bob", "city": "LA", "zip": "90001"}]

Solution:

```
import json
def flatten_users(data):
    """Flatten nested user JSON."""
    flattened = []
    for user in data.get('users', []):
        flat_user = { 'id': user['id'], 'name': user['name'], 'city': user.get('address', {}).get('city'), 'zip': user.get('address', {}).get('zip') }
        flattened.append(flat_user)
    return flattened
# Alternative: List comprehension (more Pythonic)
def flatten_users_compact(data):
    return [ { 'id': u['id'], 'name': u['name'], 'city': u.get('address', {}).get('city'), 'zip': u.get('address', {}).get('zip') } for u in data.get('users', []) ]
# Usage with open('users.json', 'r') as f: data = json.load(f)
result = flatten_users(data)
```

Key Points: • Use `.get()` to handle missing keys (avoids `KeyError`) • List comprehensions are Pythonic but readable version is fine too • Always validate input (check if 'users' key exists)

PYTHON Q2: Process Large CSV in Chunks

Problem: Read a large CSV file (100GB+) and calculate average salary by department without loading entire file into memory. **CSV Format:** employee_id,name,department,salary 1,Alice,Engineering,90000 2,Bob,Sales,80000 ...

Solution:

```
import pandas as pd from collections import defaultdict def calculate_avg_salary_by_dept(filepath,
chunksize=100000): """Calculate average salary by department using chunked reading.""" # Accumulators
dept_totals = defaultdict(float) dept_counts = defaultdict(int) # Read file in chunks for chunk in
pd.read_csv(filepath, chunksize=chunksize): # Group by department and sum for dept, group in
chunk.groupby('department'): dept_totals[dept] += group['salary'].sum() dept_counts[dept] += len(group) #
Calculate averages avg_salaries = { dept: dept_totals[dept] / dept_counts[dept] for dept in dept_totals }
return avg_salaries # Alternative: Pure Python (no pandas) def calculate_avg_salary_pure_python(filepath):
"""Using csv module for memory efficiency.""" import csv dept_totals = defaultdict(float) dept_counts =
defaultdict(int) with open(filepath, 'r') as f: reader = csv.DictReader(f) for row in reader: dept =
row['department'] salary = float(row['salary']) dept_totals[dept] += salary dept_counts[dept] += 1 return
{ dept: dept_totals[dept] / dept_counts[dept] for dept in dept_totals } # Usage result =
calculate_avg_salary_by_dept('employees.csv') print(result) # {'Engineering': 92500.0, 'Sales': 75000.0}
```

Interview Tips: • Mention chunked reading shows understanding of memory constraints • defaultdict simplifies accumulation logic • Pure Python solution shows versatility (when pandas unavailable)

PYTHON Q3: Deduplicate Records with Custom Logic

Problem: Given a list of user records with duplicates (same email), keep only the most recent record (highest timestamp). **Input:** users = [{"email": "alice@ex.com", "name": "Alice", "timestamp": 1000}, {"email": "bob@ex.com", "name": "Bob", "timestamp": 2000}, {"email": "alice@ex.com", "name": "Alicia", "timestamp": 3000}, # Latest Alice]
Output: [{"email": "bob@ex.com", "name": "Bob", "timestamp": 2000}, {"email": "alice@ex.com", "name": "Alicia", "timestamp": 3000}]

Solution:

```
def deduplicate_users(users): """Keep most recent record per email.""" # Dictionary to store latest record per email latest_users = {} for user in users: email = user['email'] # If email not seen or this record is newer if email not in latest_users or user['timestamp'] > latest_users[email]['timestamp']: latest_users[email] = user return list(latest_users.values()) # Alternative: Using max with key (more Pythonic) from itertools import groupby def deduplicate_users_v2(users): """Group by email and take max timestamp.""" # Sort by email first (required for groupby) sorted_users = sorted(users, key=lambda x: x['email']) result = [] for email, group in groupby(sorted_users, key=lambda x: x['email']): # Get record with max timestamp latest = max(group, key=lambda x: x['timestamp']) result.append(latest) return result # Alternative: Using pandas (if allowed) import pandas as pd def deduplicate_users_pandas(users): df = pd.DataFrame(users) # Sort by timestamp, keep last occurrence per email return df.sort_values('timestamp').drop_duplicates(subset='email', keep='last').to_dict('records') # Usage users = [ {"email": "alice@ex.com", "name": "Alice", "timestamp": 1000}, {"email": "bob@ex.com", "name": "Bob", "timestamp": 2000}, {"email": "alice@ex.com", "name": "Alicia", "timestamp": 3000}, ] result = deduplicate_users(users) print(result)
```

PYTHON Q4: API Pagination and Rate Limiting

Problem: Fetch all records from a paginated API with rate limiting (max 10 requests/sec). **API Response:** { "data": [...], "next_page": "https://api.example.com/users?page=2", "has_more": true }

Solution:

```
import requests
import time
from typing import List, Dict

def fetch_all_pages(base_url: str, max_requests_per_sec: int = 10) -> List[Dict]:
    """Fetch all pages from paginated API with rate limiting."""
    all_data = []
    next_url = base_url
    request_times = []
    while next_url:
        # Rate limiting logic
        now = time.time()
        # Remove timestamps older than 1 second
        request_times = [t for t in request_times if now - t < 1]
        if len(request_times) >= max_requests_per_sec:
            # Wait until we can make next request
            sleep_time = 1 - (now - request_times[0])
            if sleep_time > 0:
                time.sleep(sleep_time)
        # Make request
        try:
            response = requests.get(next_url, timeout=30)
            response.raise_for_status()
            data = response.json()
            all_data.extend(data.get('data', []))
            # Track request time
            request_times.append(time.time())
            # Get next page
            next_url = data.get('next_page')
        except requests.exceptions.RequestException as e:
            print(f"Error fetching {next_url}: {e}")
            break
    return all_data

# Alternative: Using backoff for retries
import backoff
@backoff.on_exception(backoff.expo, requests.exceptions.RequestException, max_tries=3)
def fetch_page_with_retry(url):
    response = requests.get(url, timeout=30)
    response.raise_for_status()
    return response.json()

# Usage
users = fetch_all_pages('https://api.example.com/users?page=1')
print(f"Fetches {len(users)} users")
```

PART 3: PYSPARK CODING QUESTIONS

What Interviewers Look For in PySpark:

- Understanding transformations vs actions
- Avoiding unnecessary shuffles (use broadcast joins when appropriate)
- Proper use of partitioning (repartition vs coalesce)
- Window functions for analytics
- Handling skewed data
- Caching strategies
- Performance optimization mindset

PYSPARK Q1: Deduplication with Window Function

Problem: Remove duplicate records, keeping the latest record based on timestamp. **Input DataFrame:**

id	email	name	timestamp
1	alice@ex.com	Alice	2024-01-01
2	bob@ex.com	Bob	2024-01-02
3	alice@ex.com	Alicia	2024-01-03

(keep this)

Solution:

```
from pyspark.sql import SparkSession
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

# Create Spark session
spark = SparkSession.builder.appName("dedup").getOrCreate()

# Sample data
data = [
    (1, "alice@ex.com", "Alice", "2024-01-01"),
    (2, "bob@ex.com", "Bob", "2024-01-02"),
    (3, "alice@ex.com", "Alicia", "2024-01-03"),
]
df = spark.createDataFrame(data, ["id", "email", "name", "timestamp"])

# Solution 1: Using window function
window_spec = Window.partitionBy("email").orderBy(col("timestamp").desc())
deduped_df = df.withColumn("row_num", row_number().over(window_spec)) \
    .filter(col("row_num") == 1) \
    .drop("row_num")

# Solution 2: Using dropDuplicates (simpler but less control)
deduped_df = df.orderBy(col("timestamp").desc()) \
    .dropDuplicates(["email"])

# Solution 3: Using aggregation (for keeping first/last)
from pyspark.sql.functions import max, struct
deduped_df = df.groupBy("email") \
    .agg(max(struct("timestamp", "id", "name")).alias("max_struct")) \
    .select("email", "max_struct.*")
deduped_df.show()
```

Performance Tip:

- `dropDuplicates()` is simpler but has less control over tie-breaking
- Window function gives explicit control (e.g., latest by timestamp, then highest id)
- Both trigger a shuffle (group by email)

PYSPARK Q2: Broadcast Join for Performance

Problem: Join a large transactions table with a small lookup table (e.g., product names). Optimize for performance.

Large table: transactions (billions of rows) **Small table:** products (thousands of rows)

Solution:

```
from pyspark.sql.functions import broadcast # Sample data transactions =
spark.read.parquet("s3://bucket/transactions/") products = spark.read.parquet("s3://bucket/products/") #
BAD: Regular join (shuffle both sides) result_bad = transactions.join(products, "product_id") # GOOD:
Broadcast join (send small table to all executors) result_good = transactions.join(broadcast(products),
"product_id") # When to use broadcast: # - Small table < 10 MB (configurable:
spark.sql.autoBroadcastJoinThreshold) # - Avoids shuffle of large table # - Significant performance gain #
Alternative: Spark auto-detects if table is small # But explicit broadcast() shows interviewer you
understand optimization result_good.write.parquet("s3://bucket/output/")
```

Interview Discussion Points: • Broadcast join sends small table to all executors (no shuffle) • Default threshold: 10 MB (spark.sql.autoBroadcastJoinThreshold) • Trade-off: Network overhead of broadcasting vs shuffle cost • Don't broadcast if table > executor memory • Use .explain() to verify broadcast hash join in plan

PYSPARK Q3: Calculate Running Total with Window Function

Problem: Calculate cumulative sum of sales by date within each store. **Input:** | store_id | date | sales |
|-----|-----|-----| | 1 | 2024-01-01 | 100 | | 1 | 2024-01-02 | 150 | | 2 | 2024-01-01 | 200 |

Solution:

```
from pyspark.sql.window import Window from pyspark.sql.functions import sum as _sum, col # Create sample
data data = [ (1, "2024-01-01", 100), (1, "2024-01-02", 150), (1, "2024-01-03", 120), (2, "2024-01-01",
200), (2, "2024-01-02", 180), ] df = spark.createDataFrame(data, ["store_id", "date", "sales"]) # Window
spec: partition by store_id, order by date # ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW = running
total window_spec = Window.partitionBy("store_id") \ .orderBy("date") \
.rowsBetween(Window.unboundedPreceding, Window.currentRow) # Calculate running total result =
df.withColumn("running_total", _sum("sales").over(window_spec)) result.show() # Output: #
+-----+-----+-----+-----+ # |store_id| date|sales|running_total| #
+-----+-----+-----+-----+ # | 1|2024-01-01| 100| 100| # | 1|2024-01-02| 150| 250| # |
1|2024-01-03| 120| 370| # | 2|2024-01-01| 200| 200| # | 2|2024-01-02| 180| 380| #
+-----+-----+-----+-----+ # Alternative: Without explicit window frame (same result for
SUM) window_spec_simple = Window.partitionBy("store_id").orderBy("date") result =
df.withColumn("running_total", _sum("sales").over(window_spec_simple))
```

PYSPARK Q4: Handle Data Skew

Problem: You're joining two tables but one key (e.g., `user_id = NULL`) has billions of rows causing skew. How do you handle it? **Symptom:** One task takes hours while others finish in minutes.

Solution:

```
from pyspark.sql.functions import col, rand, concat, lit # Assume 'users' and 'events' tables
users = spark.read.parquet("s3://bucket/users/")
events = spark.read.parquet("s3://bucket/events/")
# BAD: Direct join with skewed key
result = events.join(users, "user_id")
# NULL user_id causes skew
# SOLUTION 1: Filter out skewed keys, process separately
non_null_events = events.filter(col("user_id").isNotNull())
null_events = events.filter(col("user_id").isNull())
# Join non-null normally
result_non_null = non_null_events.join(users, "user_id")
# Process null separately (no join needed, or left join with broadcast)
result_null = null_events
# Handle separately
# Union results
result = result_non_null.union(result_null)
# SOLUTION 2: Salting (add random suffix to distribute skewed key)
from pyspark.sql.functions import concat, lit, rand, floor
# Add salt to events (0-9 suffix)
events_saltered = events.withColumn("user_id_saltered", concat(col("user_id"), lit("_"), floor(rand() * 10).cast("int")))
# Explode users with same salts (0-9)
from pyspark.sql.functions import explode, array, lit as _lit
salts = array(*[_lit(i) for i in range(10)])
users_exploded = users.withColumn("salt", explode(salts)) \
    .withColumn("user_id_saltered", concat(col("user_id"), lit("_"), col("salt")))
# Join on salted key (distributes load)
result = events_saltered.join(users_exploded, "user_id_saltered")
# SOLUTION 3: Adaptive Query Execution (AQE) - Spark 3.0+
# Enable AQE (auto-handles skew)
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
result = events.join(users, "user_id")
# Spark auto-detects and handles skew
```

Interview Discussion:

- Skew = uneven distribution of data across partitions
- Symptom: One executor takes 10x longer than others
- Solutions: (1) Filter + separate processing, (2) Salting, (3) AQE (Spark 3.0+)
- Always mention checking skew in `.explain()` output or Spark UI

PYSPARK Q5: Optimize Small File Problem

Problem: Your ETL job writes 10,000 small files (1MB each) instead of optimal ~128MB files. How do you fix this?

Issue: Too many small files slow down reads and overwhelm HDFS/S3 metadata.

Solution:

```
# BAD: Writes many small files df.write.partitionBy("year", "month", "day").parquet("output/") # SOLUTION
1: Repartition before write df.repartition(100) \ .write.partitionBy("year", "month", "day") \
.parquet("output/") # SOLUTION 2: Coalesce (reduce partitions without full shuffle) # Use when reducing
partitions (e.g., 1000 → 100) df.coalesce(100) \ .write.partitionBy("year", "month", "day") \
.parquet("output/") # SOLUTION 3: Control max records per file df.write.option("maxRecordsPerFile",
100000) \ .partitionBy("year", "month", "day") \ .parquet("output/") # SOLUTION 4: Manual compaction after
write # Read existing small files old_df = spark.read.parquet("output/") # Repartition and overwrite
old_df.repartition(100) \ .write.mode("overwrite") \ .partitionBy("year", "month", "day") \
.parquet("output/") # Best practice: Aim for 128MB - 1GB per file # Formula: num_partitions =
total_size_MB / target_file_size_MB total_size = 10000 # MB target_size = 128 # MB num_partitions =
total_size // target_size # ~78 df.repartition(num_partitions).write.parquet("output/")
```

Interview Key Points: • `repartition()` = full shuffle (expensive but even distribution) • `coalesce()` = reduce partitions without shuffle (use for reducing, not increasing) • Target file size: 128MB - 1GB (HDFS block size aligned) • Too many small files = metadata overhead + slow reads • Too few large files = poor parallelism

PART 4: CODING PATTERNS & INTERVIEW TIPS

SQL Coding Patterns

1. Always use CTEs (WITH clause) for readability:

WITH filtered_data AS (...), aggregated AS (...) SELECT ...

2. Window functions for ranking/running totals:

ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), SUM() OVER ()

3. JOIN explicitly (avoid implicit joins in WHERE):

Good: FROM a JOIN b ON a.id = b.id

Bad: FROM a, b WHERE a.id = b.id

4. Handle NULLs explicitly:

COALESCE(column, default_value), IS NULL, IS NOT NULL

5. Date manipulation functions:

DATE_TRUNC(), EXTRACT(), DATEADD(), DATEDIFF()

Python Coding Patterns

1. List comprehensions for transformations:

```
result = [process(x) for x in data if condition(x)]
```

2. Dictionary for lookups (O(1) vs list O(n)):

```
lookup = {item['id']: item for item in data}
```

3. Use generators for large data:

```
def process_large_file(f): yield from (line.strip() for line in f)
```

4. defaultdict for accumulation:

```
from collections import defaultdict
```

```
counts = defaultdict(int)
```

5. Context managers for files:

```
with open('file.txt') as f: data = f.read()
```

PySpark Coding Patterns

1. Avoid collect() on large data:

Use .show() for preview, .count() for validation

2. Cache frequently used DataFrames:

```
df.cache() or df.persist(StorageLevel.MEMORY_AND_DISK)
```

3. Use broadcast for small tables:

`large_df.join(broadcast(small_df), 'key')`

4. Partition data appropriately:

`df.repartition('date')` for write, `df.coalesce(10)` for reduce

5. Use Spark SQL for complex logic:

`spark.sql('SELECT ... FROM table')` is often clearer than chained DataFrame ops

INTERVIEW STRATEGY FOR CODING ROUNDS

Phase	What to Do	Time
1. Clarify	• Understand input/output format • Ask about data size (affects solution) • Clarify edge cases (NULLS, duplicates) • Confirm performance requirements	2-3 min
2. Plan	• Think out loud (show thought process) • Sketch approach on whiteboard/paper • Mention trade-offs (space vs time) • Get buy-in from interviewer	3-5 min
3. Code	• Start with simple solution (correct > optimal) • Comment complex logic • Use meaningful variable names • Write incrementally (test as you go)	15-20 min
4. Test	• Walk through with example input • Test edge cases (empty, NULL, single row) • Mention how you'd test in production	3-5 min
5. Optimize	• Discuss time/space complexity • Suggest improvements (indexes, caching) • Mention monitoring/observability	3-5 min

Common Mistakes to Avoid:

- Jumping into code without clarifying requirements
- Writing code silently (explain what you're doing)
- Ignoring edge cases (NULL, empty, duplicates)
- Not testing your code (walk through with example)
- Using `.collect()` on large PySpark DataFrames
- Writing inefficient SQL (Cartesian products, missing indexes)
- Not discussing trade-offs (time vs space, simplicity vs performance)

Things That Impress Interviewers:

- Asking clarifying questions (shows you think before coding)
- Discussing multiple approaches (shows breadth)
- Mentioning performance implications (broadcast joins, partitioning)
- Thinking about production (error handling, logging, monitoring)
- Clean, readable code (comments, meaningful names)
- Testing edge cases proactively
- Discussing observability (how would you monitor this in prod?)

QUICK REFERENCE CHEAT SHEET

SQL Window Functions:

```
-- Ranking ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) -- 1,2,3,4 RANK() OVER (ORDER BY salary DESC) -- 1,2,2,4 (skips) DENSE_RANK() OVER (ORDER BY salary DESC) -- 1,2,2,3 (no skip) -- Analytics LAG(salary) OVER (PARTITION BY dept ORDER BY date) -- Previous row LEAD(salary) OVER (PARTITION BY dept ORDER BY date) -- Next row SUM(amount) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) -- Running total AVG(salary) OVER (PARTITION BY dept) -- Department average
```

Python One-Liners:

```
# Dictionary from list lookup = {item['id']: item for item in data} # Flatten nested list flat = [item for sublist in nested_list for item in sublist] # Count occurrences from collections import Counter counts = Counter(my_list) # Group by key from itertools import groupby groups = {k: list(v) for k, v in groupby(sorted(data, key=lambda x: x['key']), key=lambda x: x['key'])} # Read JSON import json with open('file.json') as f: data = json.load(f)
```

PySpark Essential Functions:

```
from pyspark.sql.functions import col, lit, when, sum, count, avg, max, min from pyspark.sql.window import Window # Filter df.filter(col('age') > 30) # Add/modify column df.withColumn('new_col', col('old_col') * 2) # Conditional df.withColumn('category', when(col('age') < 18, 'minor').otherwise('adult')) # Aggregation df.groupBy('dept').agg(avg('salary').alias('avg_salary')) # Window function window = Window.partitionBy('dept').orderBy('salary') df.withColumn('rank', row_number().over(window)) # Join df1.join(df2, 'key', 'inner') # or 'left', 'right', 'outer'
```

Final Tips for Tomorrow:

- **Stay calm:** It's okay to ask for clarification or take a moment to think
- **Think out loud:** Interviewers want to see your thought process
- **Start simple:** Get a working solution first, optimize later
- **Test your code:** Walk through with an example
- **Discuss trade-offs:** Show you understand there are multiple approaches
- **Production mindset:** Mention error handling, logging, monitoring

Good luck! You've got this! ■